



UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

Examen Final

Curso: Inteligencia Artificial (CC421)

Semestre 2025-II (17/12/2025)

Pregunta 1. Demostración del Gradiente Local en Backpropagation (4.0 puntos)

1. Aplicación de la Regla de la Cadena (1.5 puntos)

El error E depende de la entrada neta V_J , y V_J depende del peso W_{JI} . Utilizamos la Regla de la Cadena para enlazar estas dependencias:

$$\frac{\partial E}{\partial W_{JI}} = \frac{\partial E}{\partial V_J} \cdot \frac{\partial V_J}{\partial W_{JI}}$$

(0.5 puntos por la fórmula. 0.5 puntos por identificar la secuencia de dependencia $E \rightarrow V_J \rightarrow W_{JI}$).

2. Cálculo de la Dependencia Local (1.0 punto)

La entrada neta a la neurona J es la suma ponderada de todas las salidas de la capa anterior I :

$$V_J = \sum_I W_{JI} Y_I$$

Para calcular la derivada parcial con respecto a un solo peso W_{JI} , consideramos que Y_I es una constante (la salida de la neurona I), y todos los demás términos en la sumatoria son cero o irrelevantes en la derivación parcial:

$$\frac{\partial V_J}{\partial W_{JI}} = \frac{\partial}{\partial W_{JI}} \left(W_{JI} Y_I + \sum_{K \neq I} W_{JK} Y_K \right) = Y_I$$

(Se requiere que el estudiante demuestre que el resultado de la derivada es simplemente Y_I , que es la salida de la neurona I).

3. Definición y Justificación del Gradiente Local (1.5 puntos)

Definición de Δ_J (Gradiente Local): El gradiente local Δ_J se define para simplificar la primera parte de la derivada del error total:

$$\Delta_J = -\frac{\partial E}{\partial V_J}$$

(Se añade el signo negativo porque el algoritmo de backpropagation busca el descenso por gradiente, es decir, moverse en la dirección **opuesta** al gradiente del error).

Justificación de la Eficiencia: Al sustituir los resultados de (1) y (2) en la regla de actualización de pesos generalizada ($\Delta W = -\eta \frac{\partial E}{\partial W}$), obtenemos:

$$\Delta W_{JI} = \eta \cdot \left(-\frac{\partial E}{\partial V_J} \right) \cdot Y_I$$

$$\Delta W_{JI} = \eta \cdot \Delta_J \cdot Y_I$$

La definición de Δ_J es fundamental porque resuelve el Problema de Asignación de Crédito de forma **local**. La neurona J calcula Δ_J una única vez, encapsulando cuánto afecta su entrada neta V_J al error total E .

Luego, este valor Δ_J puede ser reutilizado para ajustar **todos** los pesos W_{JI} que llegan a la neurona J , multiplicándolo por la entrada correspondiente Y_I . Esto evita tener que calcular $\frac{\partial E}{\partial W_{JI}}$ globalmente para cada peso, haciendo el entrenamiento computacionalmente eficiente.

(1 punto por definir Δ_J y mostrar la fórmula de actualización de pesos usando Δ_J . 0.5 puntos por justificar que Δ_J permite la eficiencia computacional al ser un valor reutilizable localmente).

Pregunta 2. Análisis Estructural y Funcional de Arquitecturas Clave 4.0 puntos, 1.0 puntos por fila)

Característica	MLP (Perceptrón Multicapa)	Autoencoder (AE)	CNN (Red Convolucional)	RBM (Máquina de Boltzmann Restringida)
Problema Principal Resuelto	Clasificación/Regresión para datos ya vectorizados (<i>Feature Vectors</i>).	Reducción de dimensionalidad/Compresión y aprendizaje de representaciones latentes.	Extracción automática de características y Clasificación de datos de alta dimensionalidad (e.g., imágenes).	Modelos Generativos: Aprende la distribución de probabilidad de los datos, $P(\text{Data})$.
Tipo de Conectividad (Densidad)	Densa (<i>Fully Connected</i>): Cada neurona se conecta a todas las neuronas de la capa siguiente.	Densa (Las capas oculta y de salida suelen estar completamente conectadas).	Dispersa (<i>Sparse</i>): Neuronas conectadas solo a regiones locales (campos receptivos).	Bipartita/Restringida: Conexiones solo entre capas V y H ; no hay conexiones intra-capa.
Uso de Compartición de Pesos (<i>Weight Sharing</i>)	No Aplica: Los pesos son independientes, lo que causa una explosión de parámetros.	Sí (Ligadura): Los pesos del decodificador (W^T) están ligados o son la transpuesta de los pesos del codificador (W).	Sí (Filtros): El mismo conjunto pequeño de pesos (el kernel) se aplica a toda la entrada, reduciendo masivamente los parámetros.	No Aplica Directamente: No hay <i>kernel</i> que se deslice, pero el modelo utiliza una matriz de pesos (W) única.
Mecanismo de Aprendizaje Principal	Backpropagation (Basado en la Regla Delta Generalizada) para minimizar el error entre Y^* y Y_{calc} .	Backpropagation (similar a MLP), pero la pérdida (error) se calcula entre la entrada X y la salida reconstruida $\hat{X}$.	Backpropagation (para ajustar los coeficientes de los filtros/kernels, sin algoritmo de aprendizaje nuevo).	Contraste Divergente (<i>Contrastive Divergence</i>), ya que no existe un error de salida tradicional (Y^*), sino que se minimiza la Función de Energía.

Solución 4: Concepto Teórico sobre RBM y Autoencoders

1. RBM: Naturaleza y Objetivo Principal

¿Que tipo de red neuronal es una **Máquina de Boltzmann Restringida (RBM)** y cuál es su objetivo principal de aprendizaje, en términos de la distribución de probabilidad de los datos?

- Una RBM es una **red neuronal artificial estocástica generativa** (o un modelo probabilístico) que puede aprender una **distribución de probabilidad** sobre su conjunto de entradas. Originalmente fueron propuestas bajo el nombre de **Harmonium**.

2. RBM: Función de Energía

Explique por qué las RBM se consideran modelos basados en energía y qué indica un valor escalar de energía bajo en el contexto de la configuración del modelo y la probabilidad.

- Las RBM son **modelos basados en energía** (EBM) que asocian una energía escalar a cada estado o configuración de las variables.
- Un valor de energía bajo corresponde a una **mayor probabilidad** de que el modelo esté en esa configuración. El modelo intenta minimizar la función de energía.

3. RBM: Algoritmo de Entrenamiento

¿Cuál es el algoritmo más utilizado para entrenar las RBM y en qué se basa, a diferencia de la retropropagación utilizada en las redes *feedforward*?

- El algoritmo más común para entrenar las RBM es la **Divergencia Contrastiva (CD)**.
- CD es un método de aprendizaje que se basa en la **aproximación del gradiente de log-verosimilitud** y utiliza el **muestreo de Gibbs** para calcular la actualización de pesos.
- La actualización de pesos se calcula a partir de la diferencia entre la expectativa bajo la distribución de los datos y la expectativa bajo la distribución de la reconstrucción (o *confabulation*).

4. Autoencoders: Clasificación de Aprendizaje y Pérdida

Aunque los Autoencoders se entrenan usando métodos de aprendizaje supervisado (como la retropropagación), ¿dentro de qué categoría principal de aprendizaje se clasifican y por qué? Además, mencione las funciones de pérdida comunes utilizadas.

- Los Autoencoders se clasifican como modelos de **aprendizaje no supervisado**, aunque técnicamente se entrenan utilizando métodos de aprendizaje supervisado, por lo que a menudo se denominan **auto-supervisados** (*self-supervised*).
- El aprendizaje es auto-supervisado porque el modelo intenta recrear la entrada.
- Las funciones de pérdida comunes para cuantificar la diferencia entre la entrada original (x) y la reconstrucción ($\hat{x}$) son el **error cuadrático medio (MSE)** o la **entropía cruzada** (*cross-entropy*) para datos binarios.

5. Autoencoders Profundos y RBM

Históricamente, ¿por qué fue difícil optimizar redes de Autoencoders no lineales con múltiples capas ocultas (Autoencoders profundos), y cómo utilizó Geoffrey Hinton las RBM para solucionar este problema y permitir el avance del *deep learning*?

- La dificultad radicaba en que, con pesos iniciales pequeños, los **gradientes en las capas iniciales eran diminutos** (problema del gradiente desvaneciente), lo que hacía inviable entrenar redes profundas.
- Hinton y sus colaboradores introdujeron un procedimiento de **preentrenamiento capa por capa**. Cada capa de características se entrenaba primero como una **RBM** de forma no supervisada.
- Después del preentrenamiento, las RBM se “desenrollaban” para crear un Autoencoder profundo, y los pesos obtenidos servían como una **buena inicialización** antes de la etapa de **ajuste fino** (*fine-tuning*) utilizando retropropagación.

Pregunta 4. Análisis Estructural y Gradiente en Redes Neuronales Convolucionales (7.0 puntos)

Parte I: Demostración de la Eficiencia Paramétrica (3 puntos)

1. Cálculo MLP (1 pto)

El MLP está completamente conectado.

- Entradas: $5 \times 5 = 25$ inputs.
- Salidas: $3 \times 3 \times 10 = 90$ neuronas de salida.
- Número de pesos = Entradas $\times$ Salidas (excluyendo sesgos):

$$25 \times 90 = 2250 \text{ pesos}$$

2. Cálculo CNN (1 pto)

La CNN utiliza compartición de pesos, donde cada filtro tiene un único conjunto de pesos que se aplica repetidamente.

- Tamaño del filtro: $3 \times 3 = 9$ pesos.
- Número de filtros: 10.
- Número total de pesos ajustables:

$$10 \text{ filtros} \times 9 \text{ pesos/filtro} = 90 \text{ pesos}$$

3. Justificación (1 pto)

La MLP requiere 2250 pesos, mientras que la CNN solo requiere 90. Esto representa una reducción de más del 95 % en los parámetros. La CNN es fundamentalmente más adecuada porque resuelve el problema de la **intratabilidad** al reducir drásticamente el número de parámetros a ajustar. Esta reducción permite que el algoritmo de aprendizaje (*backpropagation*) se complete en un tiempo razonable, haciendo posible el entrenamiento con conjuntos de datos masivos.

Parte II: Profundidad y Reducción Dimensional (2 puntos)

1. Función de Profundidad (1 pto)

Las CNNs utilizan capas cascada de convolución y *pooling* (similar al sistema cascada de células simples y complejas en la visión biológica).

- **Filtros múltiples:** Es fundamental emplear muchos filtros (e.g., 32, 64, 128) porque cada filtro aprende a detectar una **característica específica** (como un borde vertical, un borde horizontal, o una esquina).
- **Abstracción jerárquica:** La profundidad permite la **abstracción jerárquica**. Las capas iniciales extraen características simples (bordes). Las capas subsiguientes combinan estas características simples en patrones más complejos (formas, texturas), y las capas profundas combinan estos patrones para reconocer objetos completos (rostros, animales).

2. Configuración de Pooling (1 pto)

La operación de *pooling* (muestreo descendente) reduce el tamaño espacial de los mapas de características (reducción de dimensionalidad). Si se usa **Max Pooling**:

- **Robustez:** Al seleccionar el valor máximo de una región (ventana de *pooling*), el modelo retiene la activación más fuerte de la característica ignorando su posición exacta dentro de esa ventana. Esto hace que el modelo sea más **robusto a pequeñas traslaciones** o distorsiones de la imagen.
- **Gestión de Dimensionalidad:** Esta operación comprime la información de la capa de convolución, contrarrestando el aumento de tamaño espacial que ocurre después de la convolución y permitiendo que la red siga siendo eficiente.

Parte III: Retropropagación del Gradiente en la CNN (2 puntos)

1. Gradiente del Peso Compartido (1.0 pto)

La Regla Delta se aplica ajustando los pesos W en la dirección opuesta al gradiente total del error.

- **Necesidad de Suma:** Dado que el filtro W_{filter} es **compartido** y se aplica a múltiples ubicaciones $(l_1, l_2, \dots, l_k)$ en la imagen, este mismo conjunto de pesos contribuye a *múltiples* errores locales en el mapa de características.
- **Mecanismo:** El gradiente total del error con respecto al filtro W_{filter} es la **suma de los gradientes locales** calculados en cada posición donde el filtro fue aplicado:

$$\frac{\partial E}{\partial W_{filter}} = \sum_k \left(\frac{\partial E}{\partial W_{filter}} \right)_{\text{en posición } l_k}$$

Esta suma asegura que el ajuste de W_{filter} considere la “culpa” acumulada por ese filtro en todas las regiones de la imagen donde se utilizó. Sin esta suma, no se aprovecharía el principio de compartición de pesos para el aprendizaje, y el ajuste no sería representativo del error global.

2. Gradiente a través de Pooling (1 pto)

La operación de **Max Pooling** es no lineal y no tiene una inversa trivial. Para propagar el error hacia atrás:

- El error (Δ_J) que llega a la capa de *pooling* se distribuye a la capa de convolución anterior siguiendo la regla del *switch*.
- El error solo se propaga a la **ubicación específica** dentro de la ventana de *pooling* (ej. 2×2) que fue responsable de generar el valor máximo (el “ganador”) durante el paso de *feed-forward*. Todas las demás neuronas en la ventana de *pooling* reciben un gradiente de cero. Esto simplifica el flujo del gradiente.